

Exercice 1 – Trace (7 points)

Algorithme : **Ex1**

Entrée : /

Variables :
 - tab : **tableau de 4 entiers** initialisé avec {10, 4, 33, 20}
 - i, j, temp : **entiers**

Sortie : /

Début

```

Pour i de 0 à 2 Faire
  Pour j de (i + 1) à 3 Faire
    Si tab[j] > tab[i] Alors
      temp = tab[i]
      tab[i] = tab[j]
      tab[j] = temp
    Fin Si
  Fin Pour
Fin Pour
Fin
    
```

Trace à remplir.

i	j	tab[0]	tab[1]	tab[2]	tab[3]
/	/	10	4	33	20
0	1	10	4	33	20
0	2	33	4	10	20
0	3	33	4	10	20
1	2	33	10	4	20
1	3	33	20	4	10
2	3	33	20	10	4

Que fait cet algorithme ?

=> Il trie le tableau tab par ordre décroissant (à l'aide d'un tri à bulles).

Exercice 2 – BiduleTruc (10 points)

Écrire la fonction BiduleTruc qui reçoit en paramètre deux tableaux d'entiers (tab1 et tab2) et qui renvoie un nombre. Par exemple, avec $\text{tab1} = \{4, 8, 7, 12\}$ et $\text{tab2} = \{3, 6\}$, la fonction retourne 279. Ce nombre est déterminé par le calcul suivant : $4*3 + 4*6 + 8*3 + 8*6 + 7*3 + 7*6 + 12*3 + 12*6$.

Fonction : **BiduleTruc**

Entrée : - tab1 : tableau 1D d'entiers

 - tab2 : tableau 1D d'entiers

Variables : - i, j, sommeProduit : entiers

Sortie : entier

Début

 sommeProduit = 0

Pour i **de** 0 **à** Taille(tab1) - 1 **Faire**

Pour j **de** 0 **à** Taille(tab2) - 1 **Faire**

 sommeProduit = sommeProduit + tab1[i] * tab2[j]

Fin Pour

Fin Pour

Retourner sommeProduit

Fin

Exercice 3 – Algorithme principal (3 points)

Écrire l'algorithme principal qui affiche le résultat obtenu suite à l'appel de la fonction BiduleTruc() en utilisant l'exemple fourni.

Algorithme : **Main**

Variables : - tab1 : tableau 1D d'entiers initialisé avec {4, 8, 7, 12}

 - tab2 : tableau 1D d'entiers initialisé avec {3, 6}

 - res : entier

Début

 res = **BiduleTruc**(tab1, tab2)

Afficher("résultat = " + res)

Fin